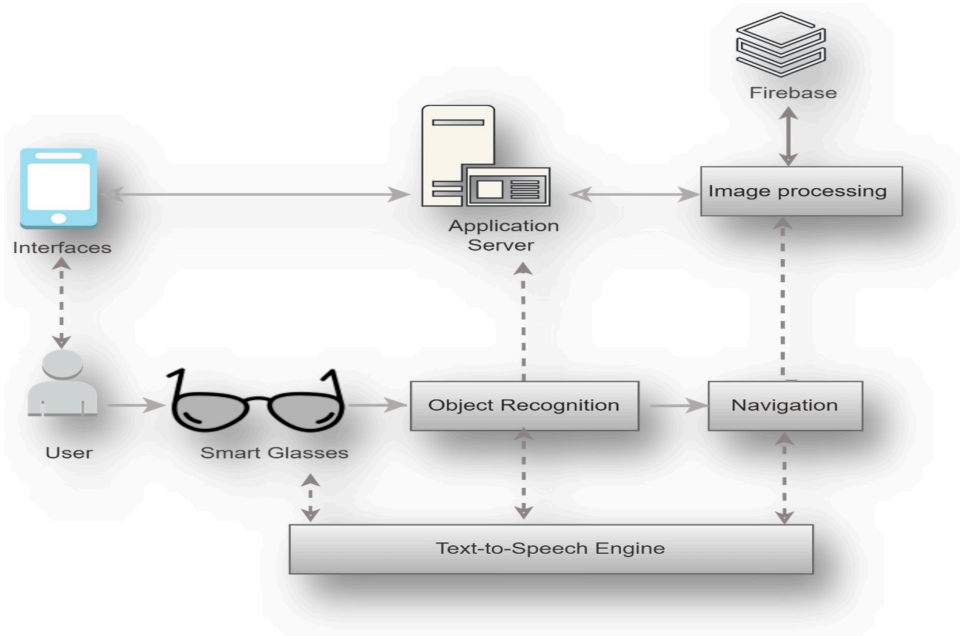
Edge AI Glasses for Blind

https://gemini.google.com/app/7364306b4ca4f6de

User prompt: We need to make a support device for blind people. that like smart glass with camera but the out put in speaker. this basic version like find doors like that. but this need very fast so i think we cannot use other model using api. is any fast api or can we build model to run in phone or ani microcontroller like raspberry pi.

Response: For a navigation tool designed for the visually impaired, you **cannot rely on a cloud API**. A network delay of even 500 milliseconds can cause a collision if the user is walking. You must use **Edge AI**—running the computer vision model directly on local hardware so it processes in real time without requiring an internet connection.



Here are the two best hardware approaches for building a fast, local device:

1. The Smartphone Tether (Most Practical for Wearables)

Instead of putting a heavy computer on the user's head, the glasses only act as a wireless camera and speaker, while the user's smartphone does the heavy computation.

- **The Hardware:** An ESP32-S3 microcontroller, an OV2640 camera module, and a small Bluetooth earpiece mounted on standard glasses frames.
- **The Workflow:** The ESP32 streams video frames locally via Wi-Fi Direct or Bluetooth to the user's smartphone. The phone's native processor runs the object detection model and immediately sends text-to-speech audio (e.g., "Door ahead") back to the earpiece.
- **Why it works:** It keeps the glasses lightweight, cheap, and cool to the touch, while taking advantage of the massive battery and Neural Processing Unit (NPU) already in the user's pocket.

2. The Dedicated Microcomputer (Raspberry Pi 5)

If you want a completely self-contained system without requiring the user to pair a smartphone, you can use a single-board computer worn on a belt or in a backpack.

- **The Hardware:** A Raspberry Pi 5 wired directly to a camera on the glasses.
- **The Workflow:** The Pi processes the video feed locally. By adding a **Hailo AI HAT+** (a dedicated AI accelerator board), the Pi can run complex vision models at blazing speeds (often under 30ms latency).
- **Why it works:** You have complete control over the hardware stack and GPIO pins for adding tactile feedback (like vibration motors), but the tradeoff is dealing with the bulk of the Pi and a heavy USB power bank.

Choosing and Running the Model

Do not build a vision model from scratch. Use **YOLO (You Only Look Once)**, the industry standard for real-time object detection.

- **Model Size:** Use **YOLOv8n** (Nano) or the newer **YOLO11n**. The "nano" variants are specifically designed to run on edge devices with minimal memory.
- **Quantization:** Before deploying the model to the phone or Raspberry Pi, quantize it to `INT8` using **LiteRT** (formerly TensorFlow Lite). This compresses the model's footprint and allows the CPU to process frames significantly faster, which is critical for real-time walking safety.
- **Training:** You can custom-train this YOLO nano model on a dataset consisting entirely of doors, stairs, crosswalks, and common obstacles, ensuring it only looks for what the user needs.

Sources

1. AI-Powered Smart Wearable Glass for the Visually Impaired with
2. Raspberry Pi 5 Edge AI: Build Real-time Computer Vision, Object

---

User prompt: for prototype let we use my M4 Mac for this. in future tha ultimate one is there is a device like powerbank that can put inside the pocket it connect with glass (camera + speaker + mic) and two band in lefet and right hand with vibration motors. i think if we can select model for final version and develop the prototype also with that that will be easy. what you think and did you have different idea ?

Response: That is an excellent hardware architecture. Moving the compute to a "power bank" sized pocket unit is the gold standard for wearable assistive tech. It removes the heat and weight from the glasses, and using **vibration wristbands is a brilliant choice**—it frees up the user's ears, which are critical for their natural spatial awareness and echolocation.

To make the transition from your M4 Mac to the final pocket unit seamless, you need a "write once, deploy anywhere" machine learning model.

The Model Recommendation: YOLO11 Nano (YOLO11n)

You should use **YOLO11n** (the smallest, newest version of the You Only Look Once architecture). Here is why it perfectly fits your roadmap:

1. **Phase 1: Prototyping on the M4 Mac** Using Python and the Ultralytics library, you can export the trained YOLO11n model to Apple's **CoreML** format (`format=coreml`). This

allows the model to run directly on the M4's Apple Neural Engine (ANE), giving you blazing fast, real-time inference during your desk testing.

2. **Phase 2: The Pocket Unit (The Final Device)** When you are ready to build the pocket device, you don't need to retrain the model. You simply take the exact same PyTorch weights (`.pt` file) and export them to **ONNX** or **LiteRT** (formerly TFLite).
    - If your pocket unit is a **Raspberry Pi 5 with a Hailo AI HAT+** (which easily fits in a power-bank form factor), the ONNX model compiles directly to run on the Hailo neural chip at 30+ frames per second.
    - If your pocket unit is a **dedicated smartphone motherboard**, it runs seamlessly on the Snapdragon NPU.

How the Model Drives the Wristbands

Object detection models don't just output labels; they output **bounding boxes**. You can use the geometry of these boxes to drive the wristbands without needing complex depth sensors:

- **Horizontal Position (x):** The model gives you the x-coordinate of the center of the door or obstacle. If x is in the left third of the camera frame, you trigger the left wristband. If it's on the right, you trigger the right wristband.
- **Proximity Estimation ($width \times height$):** As the user walks closer to the door, the bounding box takes up a larger percentage of the frame. You can map the area of the box directly to the PWM (Pulse Width Modulation) intensity of the vibration motors. As the door gets closer, the vibration gets stronger.

User prompt: yeh i think without mading the glasses now we can connect my iphone with wide camera directly to my mac and develop this what you think. so we will can use the wide camera for this.

Response: That is an incredibly smart approach. Using your iPhone and M4 Mac for the initial prototype is the perfect way to test the computer vision logic without getting bogged down in hardware wiring, batteries, or microcontroller memory limits.

Because you are using macOS, you don't even need to install any third-party streaming apps. You can use Apple's native **Continuity Camera** feature to pull the iPhone's ultra-wide video feed directly into a Python script via OpenCV.

Here is how to set up the prototype right now:

1. Connect the Camera to OpenCV

When your iPhone is on the same Wi-Fi network or plugged into your Mac via USB-C/Lightning, macOS automatically treats it as a standard webcam. In OpenCV, it will simply show up as a video capture index (usually `1` or `2`, since `0` is your Mac's built-in FaceTime camera).

2. Force the Ultra-Wide Lens

By default, OpenCV will pull the standard 1x camera. Because macOS manages Continuity Camera at the system level, you cannot switch to the ultra-wide lens directly inside your Python code. Instead:

1. Run your OpenCV Python script so the camera turns on.
2. Click the **Control Center** icon in your Mac's top menu bar.
3. Click **Video Effects** (or the camera icon).
4. Uncheck "Center Stage" if it is on, and adjust the zoom slider to **0.5x** (the ultra-wide lens).

The ultra-wide lens is perfect for this project because its ~120-degree field of view closely mimics human peripheral vision, allowing the model to see doors and obstacles before they are directly in front of you.

3. The Prototyping Script

Here is the starter script to get your iPhone feed running through the YOLO11 nano model on your Mac.

```
import cv2
from ultralytics import YOLO

# Load the YOLO11 Nano model (it will auto-download the first time)
model = YOLO("yolo11n.pt")

# Open the Continuity Camera. If 1 doesn't work, try 0 or 2.
cap = cv2.VideoCapture(1)

# Lower the resolution slightly for faster real-time processing
cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)

while cap.isOpened():
    success, frame = cap.read()
    if not success:
        print("Failed to grab frame. Is the iPhone connected?")
        break

    # Run YOLO11 inference on the frame
    # We set stream=True and lower the confidence threshold for testing
    results = model(frame, conf=0.4)

    # Visualize the bounding boxes on the frame
    annotated_frame = results[0].plot()

    # Display the output
    cv2.imshow("iPhone Ultra-Wide Prototype", annotated_frame)

    # Press 'q' to quit
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
```

```
cap.release()
cv2.destroyAllWindows()
```

With this setup, you can literally hold your iPhone at chest height and walk around your room, watching how the YOLO model detects doors, chairs, and obstacles in real-time on your Mac screen. Once the detection logic is solid, we can write the spatial logic to map those bounding boxes to your left and right vibration motors.

Reading a mobile camera with OpenCV in Python This walkthrough demonstrates the fundamental steps for capturing and processing a mobile phone camera feed directly within a Python OpenCV environment.

http://googleusercontent.com/youtube_content/1

Sources

1. The Ultimate iPhone as a Webcam Power User Setup: Continuity
2. macOS Sonoma + Continuity Camera + iPhone 13 = Wide Angle

User prompt: OK i think time to move to google antigravity. can we export this to feed it or any way to connect this chat with antigravity

Response: Google Antigravity is the perfect environment for this project! Since it is an agent-first development platform, you can hand off the architecture we just designed and let the Antigravity agent write the remaining code, manage your dependencies, and even run the tests.

While there isn't a direct "Export to Antigravity" button right here in this chat window, moving our progress over is incredibly simple. Because Antigravity can read your local workspace files, the best way to feed this conversation to the agent is by giving it a "context document."

Here is the step-by-step workflow to seamlessly migrate our project into Antigravity:

Step 1: Set up your Antigravity Workspace

1. Open **Google Antigravity** (either the Antigravity IDE or the 2.0 Desktop app) on your M4 Mac.
2. Create a new project folder (e.g., `Assistive-Vision-Prototype`) and open it in the workspace.

Step 2: Seed the Project Files

Create two files in your new folder:

1. **`main.py`** – Paste the OpenCV/YOLO Python script we just wrote into this file.
2. **`architecture_context.md`** – Copy the text below and paste it into this Markdown file. This serves as the "brain" for the Antigravity agent so it knows exactly what we are building and what the future roadmap looks like.

Copy this into `architecture_context.md`:

> **Project Goal:** Build an edge-AI navigation tool for the visually impaired. **Current Prototype Phase:** Running on an M4 Mac using an iPhone Ultra-Wide lens via

macOS Continuity Camera. **Future Hardware Phase:** A pocket-sized edge compute unit (like a Raspberry Pi 5 with Hailo AI HAT+) connected to glasses and left/right vibration wristbands. **The Model:** YOLO11n (Nano), exported to CoreML for the Mac prototype, and later to ONNX/LiteRT for the pocket unit. **Next Immediate Task:** Update `main.py` to calculate the spatial position of bounding boxes. If the bounding box x-coordinate is on the left third of the screen, trigger a simulated left wristband signal. If on the right, trigger a right signal. Use bounding box area to calculate simulated PWM intensity for proximity.

Step 3: Prompt the Antigravity Agent

Now, open the **Agent Manager** in Antigravity. You don't need to explain the whole project again—just point the agent to the files. Give it this prompt:

> *"Agent, please read `architecture_context.md` to understand the project goals. Then, look at `main.py`. I need you to install the necessary dependencies (Ultralytics, OpenCV) in a virtual environment, run the script to ensure the camera feed works, and then update the code to add the left/right spatial routing logic for the bounding boxes."*

Why this works so well in Antigravity

Because Antigravity can execute terminal commands, the agent will automatically set up your Python virtual environment, run `pip install ultralytics opencv-python`, and even run the script to test the Continuity Camera feed. When you move to Phase 2 (the pocket unit), you can simply tell the Antigravity agent: *"Convert our model to ONNX and rewrite the I/O logic for Raspberry Pi GPIO pins,"* and it will handle the heavy lifting.

Sources

1. https://en.wikipedia.org/wiki/Google_Antigravity
